

ECE276A Project 3 Report:

Visual Inertial SLAM

Stephen Huang

Department of Electrical and Computer Engineering

University of California, San Diego

La Jolla, U.S.A.

sth004@ucsd.edu

Abstract—This project report presents a visual-inertial simultaneous localization and mapping (VI-SLAM) system using an extended Kalman filter (EKF) with inertial measurement unit (IMU) and stereo camera measurements. Because IMU integration alone accumulates drift over time, visual feature observations are incorporated to both correct the estimated trajectory and landmark positions in the environment. Our approach performs EKF prediction using the IMU linear and angular velocity measurements on the robot pose with $SE(3)$ kinematics. The predicted state is then corrected through an EKF update step based on a stereo-camera observation model, where we compare the estimated landmark to its actual position to refine both the robot trajectory and the landmark estimates. This process jointly estimates the robot pose and a map of static landmarks, producing a more accurate representation of the environment.

I. INTRODUCTION

Visual-inertial simultaneous localization and mapping (VI-SLAM) estimates a robot trajectory while building a map of an unknown environment from inertial and visual sensor measurements collected over time. In many robotics settings, however, reliable localization and mapping is difficult because inertial measurements accumulate drift, while visual measurements can be noisy and incomplete due to occlusion and feature mismatch. This reduces trajectory accuracy and distorts the resulting map, which impacts tasks such as planning and navigation.

To address these limitations, we use IMU measurements to estimate the robot pose over time and use stereo visual feature observations obtained from corner detection and optical flow to correct the predicted state through an observation model. The estimated trajectory and landmark positions are updated jointly using the extended Kalman Filter (EKF). The end goal is to reduce the effect of cumulative drift and improve the consistency and accuracy of the reconstructed map.

Our approach estimates robot poses in $SE(3)$ and landmark positions in $\mathbb{R}^3$. We first compute a gaussian distribution with the EKF prediction step using IMU data and $SE(3)$ kinematics. Then, we update the predicted state using stereo features through the EKF update step. Our full process consists of:

- predicting the next pose mean by integrating the IMU measurements in $SE(3)$,

- predicting the next joint covariance with the pose perturbation,
- triangulating stereo feature observations from pixel coordinates to world coordinates,
- forming predicted feature pixel observations by projecting landmark world coordinates into pixel coordinates,
- computing the observation Jacobian with respect to the pose and Kalman gain,
- updating the pose mean through the EKF update step on $SE(3)$ with visual residuals, and
- updating the joint covariance using the EKF update step.

II. PROBLEM FORMULATION

Consider a robot navigating in an unknown, 3D environment using inertial and stereo visual measurements. We want to estimate the robot trajectory and the positions of static point landmarks observed in the environment over time. The robot is equipped with an inertial measurement unit (IMU) and a stereo camera, where the extrinsic calibration between the sensors and the stereo camera intrinsic calibration are assumed known.

A. State and Pose Representation

Let the IMU pose at time t be

$$T_t := {}_wT_{I,t} = \begin{bmatrix} R_t & p_t \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3), \quad (1)$$

where $R_t \in SO(3)$ is the IMU orientation and $p_t \in \mathbb{R}^3$ is the IMU position in the world frame. Over a period of time T , we define the unknown pose trajectory as

$$T_{0:T} := \{T_t\}_{t=0}^T. \quad (2)$$

Let the environment contain M static point landmarks with world-frame coordinates

$$m_j \in \mathbb{R}^3, \quad j = 1, \dots, M, \quad (3)$$

and define the full landmark state as

$$m := [m_1^\top \quad m_2^\top \quad \dots \quad m_M^\top]^\top \in \mathbb{R}^{3M}. \quad (4)$$

B. Sensor Measurements

At each time t , the IMU provides linear velocity $v_t \in \mathbb{R}^3$ and angular velocity $\omega_t \in \mathbb{R}^3$. We define the control input as

$$u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} \in \mathbb{R}^6. \quad (5)$$

The stereo camera provides feature observations

$$z_{t,i} \in \mathbb{R}^4, \quad i = 1, \dots, N_t, \quad (6)$$

where each observation contains the left and right image pixel coordinates of a tracked feature. Stacking all feature observations at time t gives

$$z_t := [z_{t,1}^\top \quad z_{t,2}^\top \quad \dots \quad z_{t,N_t}^\top]^\top \in \mathbb{R}^{4N_t}. \quad (7)$$

We assume the stereo calibration matrix K_s and the extrinsic transformation ${}_O T_I \in SE(3)$ from the IMU frame to the camera optical frame are known. We also assume the data association

$$\Delta_t : \{1, \dots, M\} \rightarrow \{1, \dots, N_t\} \quad (8)$$

is known, where landmark j corresponds to observation $z_{t,i}$ with $i = \Delta_t(j)$.

C. Motion Model

The IMU pose evolves according to discrete-time $SE(3)$ kinematics driven by the measured linear and angular velocities. Let τ_t denote the time increment between two consecutive measurements. The motion model is

$$T_{t+1} = T_t \exp(\tau_t \hat{u}_t), \quad (9)$$

where $\hat{u}_t \in \mathfrak{se}(3)$ is the matrix representation of the generalized velocity u_t .

Within the EKF, we represent the pose distribution using a mean $\mu_{t|t} \in SE(3)$ and perturbation $\delta\mu_{t|t} \in \mathbb{R}^6$ such that

$$T_t = \mu_{t|t} \exp(\widehat{\delta\mu_{t|t}}), \quad \delta\mu_{t|t} \sim \mathcal{N}(0, \Sigma_{t|t}), \quad (10)$$

with covariance $\Sigma_{t|t} \in \mathbb{R}^{6 \times 6}$.

D. Observation Model

Each landmark is observed through the stereo camera projection model. For landmark m_j , the corresponding feature observation satisfies

$$z_{t,i} = h(T_t, m_j) + v_{t,i} := K_s \pi({}_O T_I T_t^{-1} m_j) + v_{t,i}, \quad (11)$$

where $v_{t,i} \sim \mathcal{N}(0, V)$ is measurement noise and $\pi(\cdot)$ is the stereo projection function. Stacking all observations at time t gives

$$z_t = h(T_t, m) + v_t, \quad v_t \sim \mathcal{N}(0, I \otimes V). \quad (12)$$

E. Visual-Inertial SLAM Problem

Given IMU measurements $\{u_t\}_{t=0}^{T-1}$, stereo feature observations $\{z_t\}_{t=0}^T$, known calibration parameters K_s and ${}_O T_I$, and known correspondences $\{\Delta_t\}_{t=0}^T$, the visual-inertial SLAM problem is to jointly estimate the IMU pose trajectory $T_{0:T}$ and the landmark positions m . In particular, we seek a recursive state estimate that uses the IMU motion model to predict the next pose and the stereo observation model to update the pose and landmark estimates from visual measurements.

III. TECHNICAL APPROACH

A. IMU Localization via EKF Prediction

We begin by estimating the IMU trajectory using only the EKF prediction step with the measured linear and angular velocities. Since the pose lies in $SE(3)$ rather than Euclidean space, we represent the mean directly as a homogeneous transformation matrix.

1) *Inputs and Initialization:* Let the IMU measurements at time t be the linear velocity $v_t \in \mathbb{R}^3$ and angular velocity $\omega_t \in \mathbb{R}^3$. We stack them into the control input

$$u_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix} \in \mathbb{R}^6. \quad (13)$$

Given timestamps $\{\tau_t\}_{t=0}^{T-1}$, we define the discrete time increment

$$\Delta\tau_t = \tau_t - \tau_{t-1}, \quad (14)$$

with the initial increment taken as zero.

The EKF mean is initialized at the identity pose

$$\mu_0 = I_4, \quad (15)$$

and the initial covariance is set to

$$\Sigma_0 = \mathbf{0}_{6 \times 6}. \quad (16)$$

We use a diagonal process noise model

$$W = \text{diag}(\sigma_v^2, \sigma_v^2, \sigma_v^2, \sigma_\omega^2, \sigma_\omega^2, \sigma_\omega^2), \quad (17)$$

where

$$\sigma_v^2 = 10^{-5}, \quad \sigma_\omega^2 = 10^{-5}. \quad (18)$$

2) *Pose Mean Prediction:* To predict the mean pose in $SE(3)$, we convert each generalized velocity u_t into its twist-matrix representation

$$\hat{u}_t \in \mathfrak{se}(3). \quad (19)$$

The discrete-time prediction step for the pose mean is then

$$\mu_t = \mu_{t-1} \exp(\Delta\tau_t \hat{u}_t), \quad (20)$$

where $\exp(\cdot)$ denotes the matrix exponential. We use `scipy.linalg.expm` for this.

3) *Covariance Prediction*: To predict the uncertainty, we use the adjoint-twist matrix

$$u_t^\wedge \in \mathbb{R}^{6 \times 6}, \quad (21)$$

constructed from the same control input. The covariance is updated according to the linearized perturbation dynamics:

$$\Sigma_t = \exp(-\Delta\tau_t u_t^\wedge) \Sigma_{t-1} \exp(-\Delta\tau_t u_t^\wedge)^\top + \Delta\tau_t W. \quad (22)$$

This accounts for both the uncertainty added from the mean prediction and the added process noise at each timestep.

4) *Trajectory Storage*: After each prediction step, we store the updated mean pose

$$\{\mu_t\}_{t=0}^{T-1}, \quad \{\Sigma_t\}_{t=0}^{T-1}, \quad (23)$$

where each $\mu_t \in SE(3)$ represents the predicted IMU pose and each $\Sigma_t \in \mathbb{R}^{6 \times 6}$ represents the associated uncertainty. These predicted poses are later used as the motion prior for the visual update step.

B. Feature Detection and Matching

For the extra credit portion, we generate stereo feature tracks directly from the image sequences in dataset02 using OpenCV corner detection and optical flow. The goal is to construct feature observations of the form

$$z_t \in \mathbb{R}^{4 \times M}, \quad (24)$$

where each column corresponds to one tracked landmark and stores the left-right pixel coordinates

$$z_{t,j} = [l_{x,j} \quad l_{y,j} \quad r_{x,j} \quad r_{y,j}]^\top. \quad (25)$$

If feature j is not observed at time t , its measurement is recorded as

$$z_{t,j} = [-1 \quad -1 \quad -1 \quad -1]^\top. \quad (26)$$

1) *Corner Detection and Landmark Initialization*: We begin by detecting image features in the left camera image using Shi-Tomasi corner detection through `cv.goodFeaturesToTrack`. In the implementation, we use at most 500 candidate corners, with quality level 0.02 and minimum separation distance 10 pixels. Each new feature is assigned a unique landmark ID, and a full observation history is initialized across all timesteps. For a sequence of length T , each landmark track is initialized as

$$\ell_j = \{[-1 \quad -1 \quad -1 \quad -1]\}_{t=0}^{T-1}. \quad (27)$$

This allows all features to be stored in a common global index, even when they are absent in many frames.

2) *Optical Flow Tracking Over Time*: After detecting initial features in the first left image, we track them through time using the Lucas-Kanade optical flow algorithm with `cv.calcOpticalFlowPyrLK`. For each timestep $t > 0$, the tracked feature locations are propagated from the previous left image to the current left image. If the optical-flow status flag indicates failure, the corresponding feature is removed from the active set.

To maintain a sufficient number of tracked features, we redetect new corners whenever the number of active tracks falls below 100. These newly detected points are assigned new landmark IDs and appended to the current active feature set.

3) *Optical Flow Stereo Matching*: At each timestep, the currently active left-image features are matched to the right image using the same Lucas-Kanade optical flow. That is, for each left-image point (l_x, l_y) at time t , we estimate the corresponding right-image point (r_x, r_y) by tracking from the left image to the right image of the same stereo pair. If the optical-flow status is valid, the full stereo observation is recorded as

$$z_{t,j} = [l_{x,j} \quad l_{y,j} \quad r_{x,j} \quad r_{y,j}]^\top. \quad (28)$$

Otherwise, the feature remains marked as unobserved for that frame.

4) *Feature Matrix Construction*: After temporal tracking and stereo matching are completed, we form the feature matrix

$$Z \in \mathbb{R}^{4 \times M \times T}, \quad (29)$$

where M is the total number of landmark IDs created during the sequence. The entry $Z[:, j, t]$ stores the stereo feature observation for landmark j at timestep t , with missing observations left as -1 . This is used later in landmark mapping and VI-SLAM.

C. Landmark Mapping via EKF Update

After obtaining the predicted IMU trajectory, we estimate the 3D positions of static landmarks using an EKF update over the landmark state only. In this stage, the IMU poses are treated as known, and the visual feature observations are used to recursively initialize and refine the landmark coordinates.

1) *State, Noise Model, and Initialization*: Let the landmark state be

$$\mu_m = [m_1^\top \quad m_2^\top \quad \cdots \quad m_M^\top]^\top \in \mathbb{R}^{3M}, \quad (30)$$

where $m_j \in \mathbb{R}^3$ denotes the world-frame coordinate of landmark j . The joint covariance over all landmarks is

$$\Sigma_m \in \mathbb{R}^{3M \times 3M}. \quad (31)$$

In my implementation, the landmark mean is initialized as zeros, while the covariance is stored as a sparse matrix in `lil_matrix` format to reduce memory cost.

For the observation noise, we use

$$V = \sigma_z^2 I_4, \quad (32)$$

with

$$\sigma_z^2 = 100. \quad (33)$$

When a new landmark is first initialized, its covariance block is set to

$$\Sigma_{m_j}^{(0)} = 1000I_3. \quad (34)$$

We also maintain a Boolean initialization mask to indicate whether each landmark has already been triangulated.

2) *Stereo Intrinsic Matrix*: Given the left and right camera intrinsic matrices K_l and K_r , together with the left and right camera extrinsics relative to the IMU, we construct the stereo projection matrix

$$K_s = \begin{bmatrix} f_s u & 0 & c_u & 0 \\ 0 & f_s v & c_v & 0 \\ f_s u & 0 & c_u & f_s u b \\ 0 & f_s v & c_v & 0 \end{bmatrix}, \quad (35)$$

where b is the stereo baseline recovered from the relative left-to-right camera transform. This matrix is used in the Jacobian-based stereo observation model for the EKF update.

3) *Valid Feature Selection*: At each timestep t , we extract the feature observation matrix

$$z_t \in \mathbb{R}^{4 \times M}. \quad (36)$$

We only consider valid features with pixels that are within the 600 x 480 resolution. That is, we have

$$0 \leq l_x, r_x < 600, \quad 0 \leq l_y, r_y < 480, \quad (37)$$

with missing features marked by -1 . If no valid landmarks are observed at time t , the update step is skipped.

We also pre-process data to remove unstable or dynamic landmarks. We do this by estimating the trajectory through IMU dead-reckoning. We first integrate the IMU motion model over all timesteps to obtain a sequence of approximate poses, which are then used only for this preprocessing step.

For each landmark track, we identify the timesteps at which the feature is validly observed. We discard landmarks that are present in less than 5 frames (0.1 seconds). For the remaining landmarks, we select two observations separated by at least 100 frames (2 seconds). Using the corresponding dead-reckoned poses, we triangulate the landmark position at both timesteps using stereo triangulation. If both triangulations succeed and the two reconstructed world positions differ by more than 0.5 meters, the feature is treated as dynamic and removed from the measurement set by setting their full observation history to -1 . We also remove landmarks that are too far (15 meters away) and too close (1 meter).

4) *Landmark Initialization by Stereo Triangulation*: For landmarks observed for the first time, we initialize their world-frame coordinates through stereo triangulation using the left-right pixel disparity. Let

$$u_L, v_L, u_R \quad (38)$$

denote the left-image horizontal coordinate, left-image vertical coordinate, and right-image horizontal coordinate of a matched stereo feature. With focal lengths f_{s_u} , f_{s_v} , principal point (c_u, c_v) , and stereo baseline b , we compute the disparity

$$d = u_L - u_R. \quad (39)$$

If the disparity is non-positive, the triangulation is treated as invalid. Otherwise, the depth is recovered using

$$Z = \frac{f_{s_u} b}{d}. \quad (40)$$

The 3D point in the left camera frame is then obtained by back-projection:

$$X = \frac{(u_L - c_u)Z}{f_{s_u}}, \quad Y = \frac{(v_L - c_v)Z}{f_{s_v}}, \quad (41)$$

$$m_{C_l, j} = [X \quad Y \quad Z]^\top.$$

To reject faulty initializations, we discard points with invalid or extreme depth. That is, if

$$Z \leq 0 \quad \text{or} \quad Z > 40, \quad (42)$$

we reject it.

5) *Predicted Observation*: For landmarks that have already been initialized, we compute the predicted stereo observation using the current landmark mean and the known IMU pose at time t . Let m_j be a landmark in homogeneous form. Its coordinates in the left and right camera frames are

$$q_L = c_l T_l I T_W m_j, \quad q_R = c_r T_r I T_W m_j. \quad (43)$$

Applying the projection function $\pi(\cdot)$ and the camera intrinsics gives the predicted pixel coordinates in each image. Stacking them gives

$$\tilde{z}_{t,j} = [\tilde{l}_x \quad \tilde{l}_y \quad \tilde{r}_x \quad \tilde{r}_y]^\top. \quad (44)$$

6) *Observation Jacobian*: For each observed landmark, we compute the Jacobian of the stereo observation model with respect to the landmark coordinates. Using homogeneous coordinates and the projection Jacobian, we get

$$H_{t,j} = K_s \frac{d\pi}{dq}(T m_j) T P^\top, \quad (45)$$

where

$$T = c_l T_l I T_W, \quad P = [I_3 \quad 0]. \quad (46)$$

Each block $H_{t,j} \in \mathbb{R}^{4 \times 3}$ is inserted into a sparse global observation matrix

$$H_t \in \mathbb{R}^{4N_t \times 3M}, \quad (47)$$

where N_t is the number of currently observed initialized landmarks.

7) *EKF Landmark Update*: For the set of existing landmarks at time t , we form the stacked residuals

$$r_t = z_t - \tilde{z}_t. \quad (48)$$

We then form the S matrix

$$S_t = H_t \Sigma_m H_t^\top + I \otimes V, \quad (49)$$

and the Kalman gain is computed as

$$K_t = \Sigma_m H_t^\top S_t^{-1}. \quad (50)$$

The landmark mean is then updated by

$$\mu_{m,t+1} = \mu_{m,t} + K_t r_t, \quad (51)$$

and the covariance is updated by

$$\Sigma_{m,t+1} = (I - K_t H_t) \Sigma_{m,t}. \quad (52)$$

In the implementation, we use sparse matrices for the covariance and jacobian. This saves memory and also helps speed up computation.

D. Visual-Inertial SLAM

We now combine the IMU prediction model from Section III-A with the landmark update model from Section III-C into a joint visual-inertial SLAM estimator. Rather than estimating the IMU trajectory and landmark map separately, we maintain a single EKF state consisting of the current IMU pose and the selected landmark coordinates.

1) *Joint State and Covariance:* Let the VI-SLAM state consist of the IMU pose and a set of selected landmarks:

$$x_t = (T_t, m), \quad (53)$$

where

$$T_t \in SE(3), \quad m = [m_1^\top \quad m_2^\top \quad \cdots \quad m_{N_{\text{lm}}}^\top]^\top \in \mathbb{R}^{3N_{\text{lm}}}. \quad (54)$$

As in Section III-A, the pose mean is represented directly in $SE(3)$, while the perturbation is represented in $\mathbb{R}^6$. The full joint covariance is

$$\Sigma_t \in \mathbb{R}^{(6+3N_{\text{lm}}) \times (6+3N_{\text{lm}})}, \quad (55)$$

with structure

$$\Sigma_t = \begin{bmatrix} \Sigma_{TT} & 0 \\ 0 & \Sigma_{mm} \end{bmatrix}, \quad (56)$$

where $\Sigma_{TT} \in \mathbb{R}^{6 \times 6}$ is the pose covariance and $\Sigma_{mm} \in \mathbb{R}^{3N_{\text{lm}} \times 3N_{\text{lm}}}$ is the landmark covariance. The pose-landmark cross covariance is zero since the landmarks are assumed static. We store this using a sparse matrix.

2) *Landmark Selection:* To keep the joint EKF computationally feasible, we do not include every tracked feature in the VI-SLAM state. Instead, we first filter for valid landmarks using the approach in Section III-C. Then, from the filtered set, we select a random subset of landmarks to include in the joint state.

In the implementation, a landmark is considered eligible if it has at least one valid observation over the full sequence. Let the total number of such filtered valid landmarks be M_{valid} . We then choose

$$N_{\text{lm}} = \max(0.6M_{\text{valid}}, 1200) \quad (57)$$

landmarks uniformly at random without replacement, and retain only their corresponding feature tracks for the SLAM update. This reduces the state dimension to

$$n = 6 + 3N_{\text{lm}}, \quad (58)$$

which significantly lowers the computational cost while also having a large enough set of features for pose correction.

3) *Prediction Step for the Joint State:* The pose mean prediction is identical to the propagation described in Section III-A:

$$\mu_{T,t+1|t} = \mu_{T,t|t} \exp(\Delta \tau_t \hat{u}_t). \quad (59)$$

Since the landmarks are assumed static, their mean is unchanged during prediction:

$$\mu_{m,t+1|t} = \mu_{m,t|t}. \quad (60)$$

The covariance prediction is applied only to the pose block. Let

$$F_t = \exp(-\Delta \tau_t u_t^\wedge). \quad (61)$$

Then the predicted pose covariance is

$$\Sigma_{TT,t+1|t} = F_t \Sigma_{TT,t|t} F_t^\top + \Delta \tau_t W, \quad (62)$$

4) *Landmark Initialization:* Landmark initialization follows the same triangulation procedure as in Section III-C, where uninitialized landmarks are triangulated through stereo measurements in the current left-camera frame and then transformed into the world frame using the current predicted IMU pose. We reject points based if their depth is too extreme. For each accepted new landmark, we set its initial covariance block to

$$\Sigma_{m_j}^{(0)} = I_3, \quad (63)$$

which is larger than what was used in Section III-C.

5) *Joint Observation Jacobian:* For initialized landmarks, the stereo observation model is the same as in the landmark mapping and visual odometry formulations. However, in VI-SLAM the observation depends on both the pose and the landmark position, so the Jacobian must include both components.

The landmark Jacobian block is identical to the one used in Section III-C:

$$H_{\text{lm},i} = K_s \frac{d\pi}{dq}(q_i) T P^\top \in \mathbb{R}^{4 \times 3}, \quad (64)$$

where $q_i = T m_i$ in homogeneous coordinates.

For the pose Jacobian, we use the perturbation form:

$$H_{\text{pose},i} = -K_s \frac{d\pi}{dq}(q_i) c_i T_I (\mu_{t+1|t}^{-1} m_j)^\odot \in \mathbb{R}^{4 \times 6}. \quad (65)$$

where m_j is the world coordinates in homogenous form. The circle dot operator is defined as

$$s^\odot = \begin{bmatrix} I & -\hat{s} \\ 0 & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 6}, \quad (66)$$

For each observed landmark j , we insert both Jacobian blocks into the sparse global observation matrix

$$H_t \in \mathbb{R}^{4N_t \times (6+3N_{\text{lm}})}, \quad (67)$$

where the first six columns correspond to the pose and the remaining columns correspond to the selected landmarks. Thus, each observation row block contains a dense pose term and a single nonzero landmark block.

6) *EKF Update of Pose and Landmarks*: Using the predicted stereo observations and stacked measurement residuals

$$r_t = z_t - \tilde{z}_t, \quad (68)$$

We reject any values

$$r_t \geq 15.$$

we then use the surviving values to compute the S matrix

$$S_t = H_t \Sigma_t H_t^\top + I \otimes V, \quad (69)$$

and the Kalman gain

$$K_t = \Sigma_t H_t^\top S_t^{-1}. \quad (70)$$

Using the predicted stereo observations and stacked measurement residuals

$$r_t = z_t - \tilde{z}_t, \quad (71)$$

We reject any values

$$r_t \geq 15.$$

we then use the surviving values to compute the S matrix

$$S_t = H_t \Sigma_t H_t^\top + I \otimes V, \quad (72)$$

and the Kalman gain

$$K_t = \Sigma_t H_t^\top S_t^{-1}. \quad (73)$$

The resulting correction vector has the form

$$\delta_t = K_t r_t \in \mathbb{R}^{6+3N_{\text{lm}}}. \quad (74)$$

We split this correction into pose and landmark components:

$$\delta_t = \begin{bmatrix} \delta_{T,t} \\ \delta_{m,t} \end{bmatrix}, \quad \delta_{T,t} \in \mathbb{R}^6, \quad \delta_{m,t} \in \mathbb{R}^{3N_{\text{lm}}}. \quad (75)$$

The landmark means are updated additively,

$$\mu_{m,t|t} = \mu_{m,t|t-1} + \delta_{m,t}, \quad (76)$$

while the pose is updated on $SE(3)$ through exponential algebra,

$$\mu_{T,t+1|t+1} = \mu_{T,t+1|t} \exp(\widehat{\delta_{T,t}}). \quad (77)$$

Instead of the standard covariance correction, we use the Joseph-form update for improved numerical stability [4]:

$$\Sigma_{t+1|t+1} = (I - K_t H_t) \Sigma_{t+1|t} (I - K_t H_t)^\top + K_t (I \otimes V) K_t^\top. \quad (78)$$

We also re-symmetrize the joint covariance matrix through

$$\Sigma_{t+1|t+1} \leftarrow \frac{1}{2} (\Sigma_{t+1|t+1} + \Sigma_{t+1|t+1}^\top), \quad (79)$$

which helps reduce numerical asymmetry caused by arithmetic imprecision.

7) *Trajectory and Snapshot Storage*: After each update, we store the corrected pose in the trajectory sequence for visualization. In addition, we save intermediate trajectory-map snapshots at selected percentages of the full sequence. These snapshots are used later in the results section to visualize how the estimated path and landmark map evolve over time as the VI-SLAM updates accumulate.

IV. RESULTS

A. Results for Dataset 00

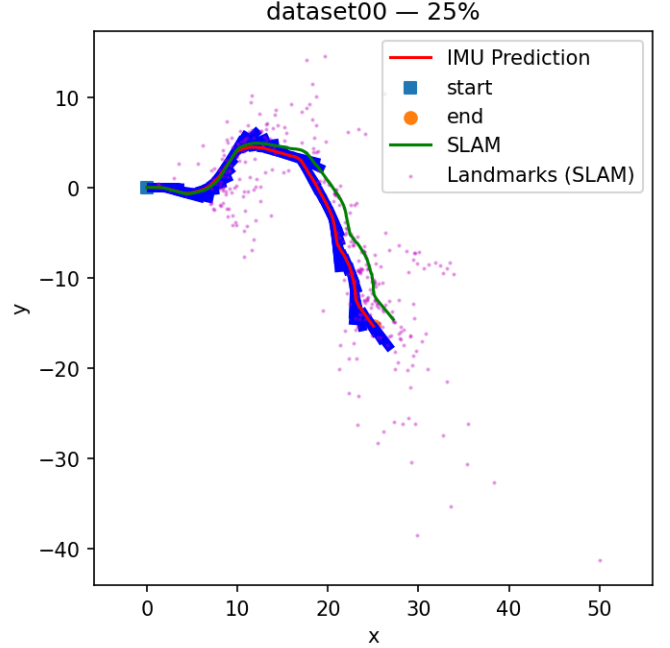


Fig. 1. Dataset 00: VI-SLAM results at 25%.

1) 25%:

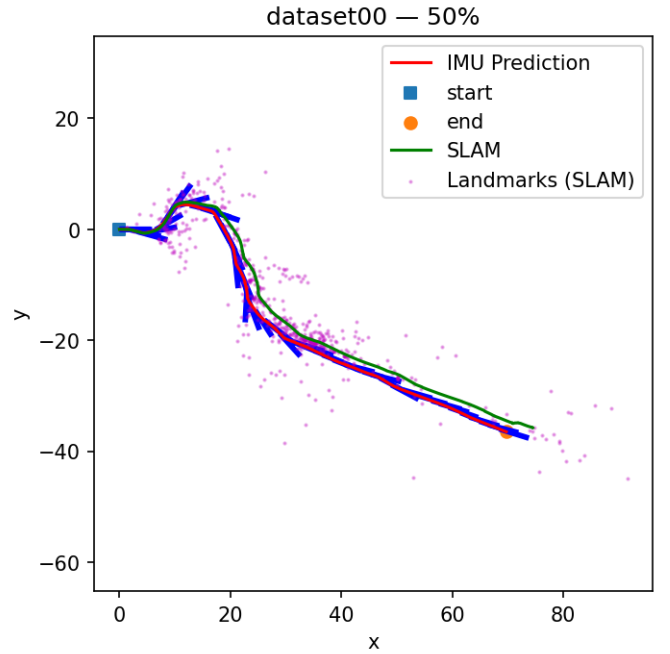


Fig. 2. Dataset 00: VI-SLAM results at 50%.

2) 50%:

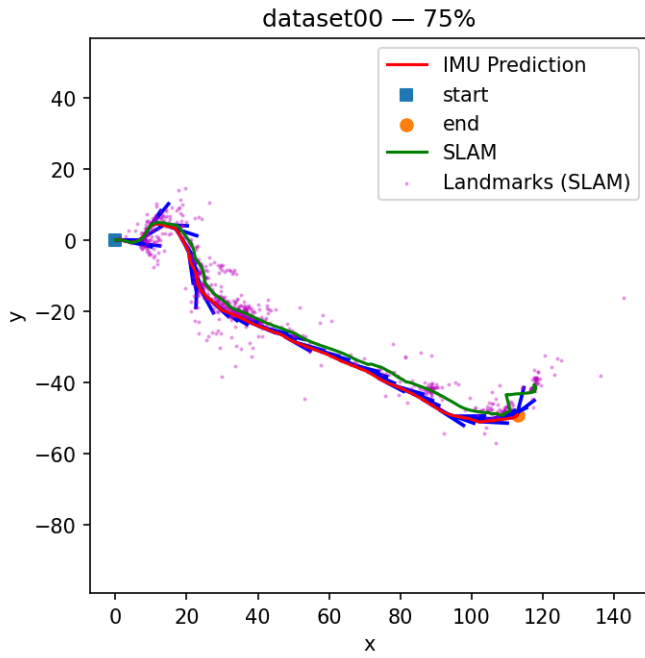


Fig. 3. Dataset 00: VI-SLAM results at 75%.

3) 75%:

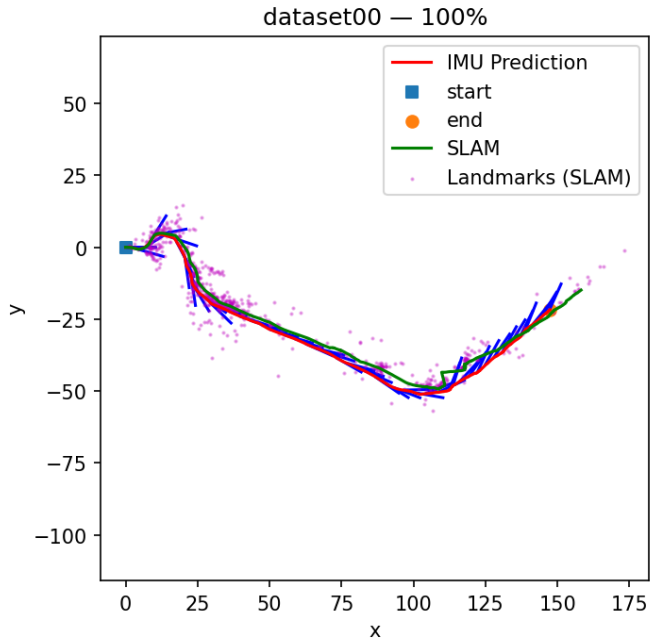


Fig. 4. Dataset 00: VI-SLAM results at 100%.

4) 100%:

B. Results for Dataset 01

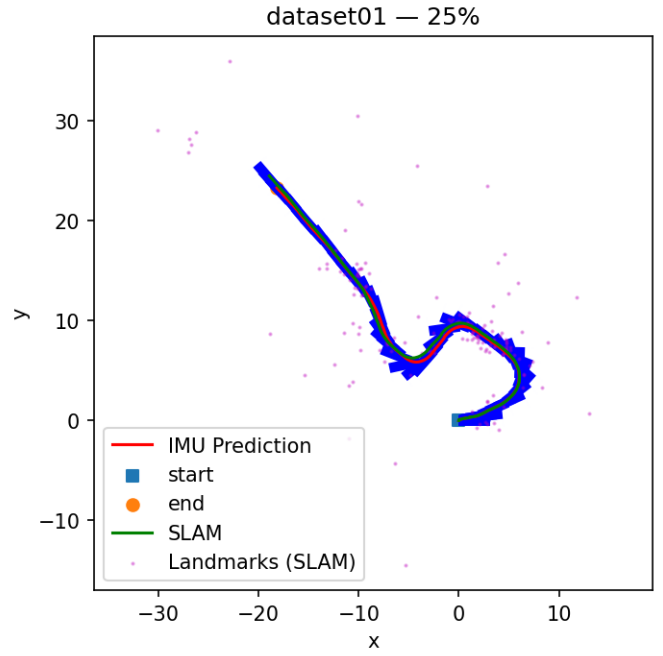


Fig. 5. Dataset 01: VI-SLAM results at 25%.

1) 25%:

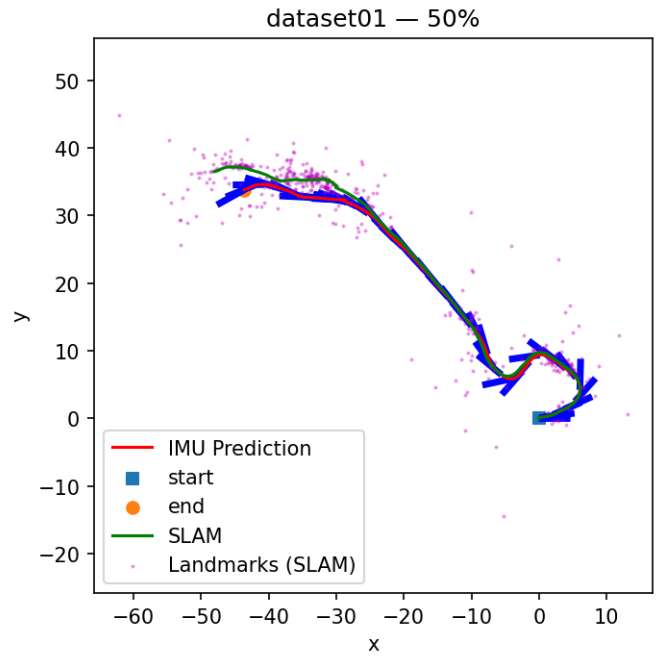


Fig. 6. Dataset 01: VI-SLAM results at 50%.

2) 50%:

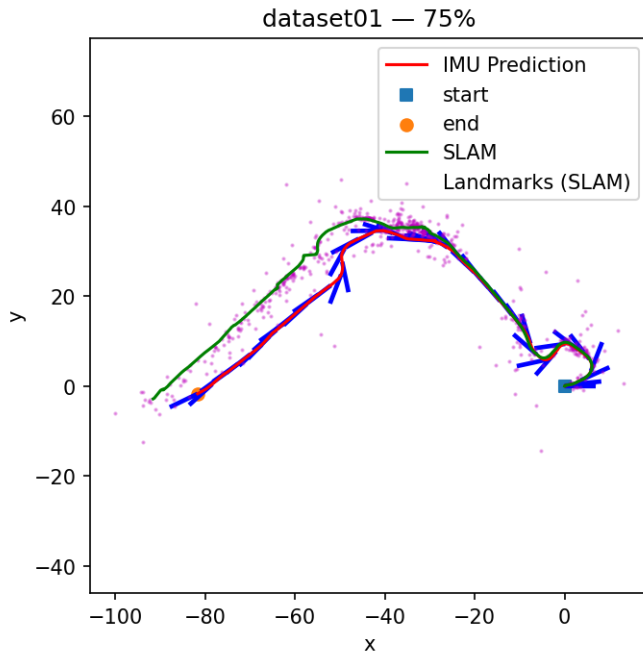


Fig. 7. Dataset 01: VI-SLAM results at 75%.

C. Results for Dataset 02

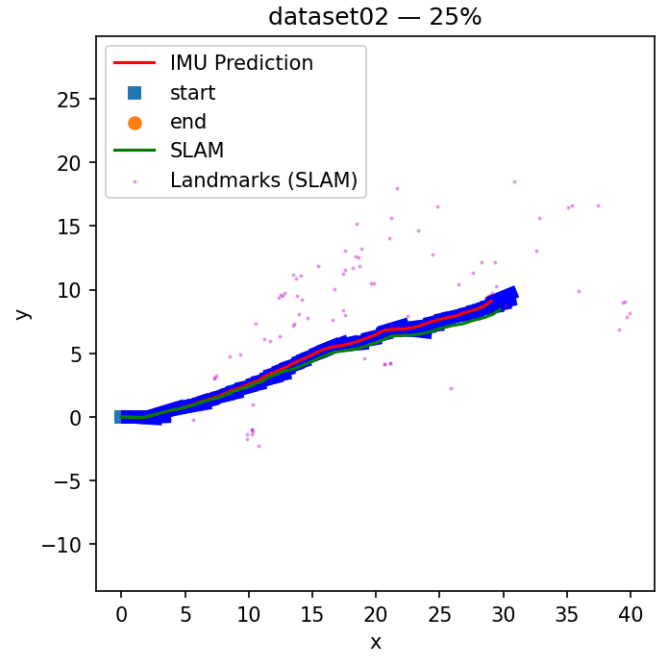


Fig. 9. Dataset 02: VI-SLAM results at 25%.

3) 75%:

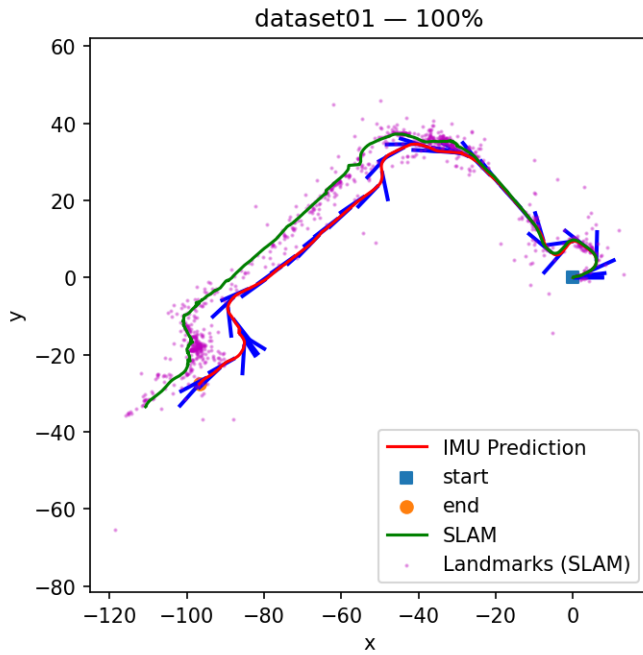


Fig. 8. Dataset 01: VI-SLAM results at 100%.

4) 100%:

1) 25%:

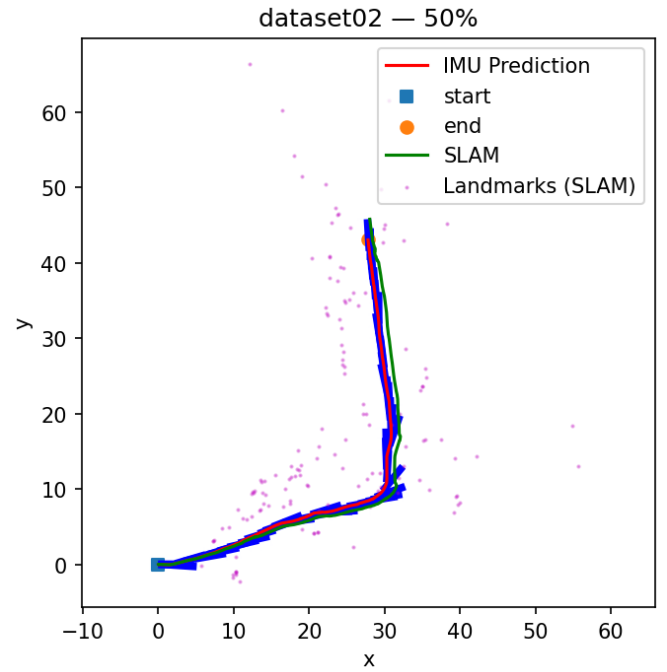


Fig. 10. Dataset 02: VI-SLAM results at 50%.

2) 50%:

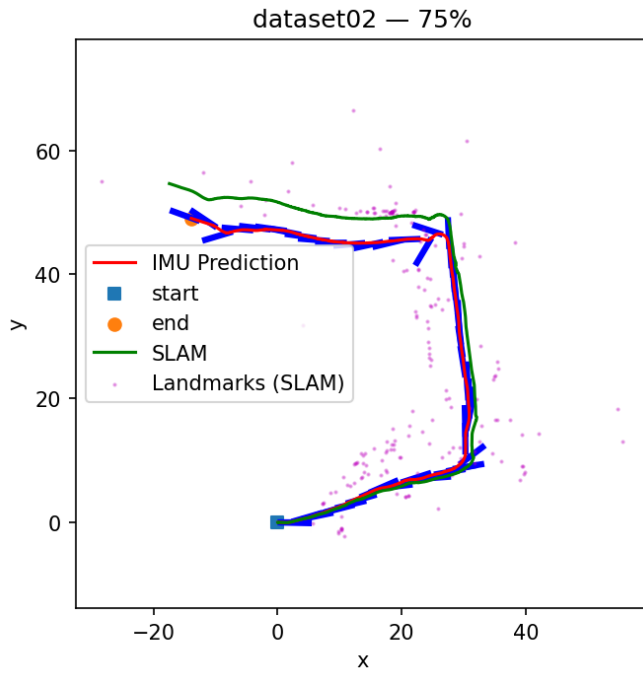


Fig. 11. Dataset 02: VI-SLAM results at 75%.

3) 75%:

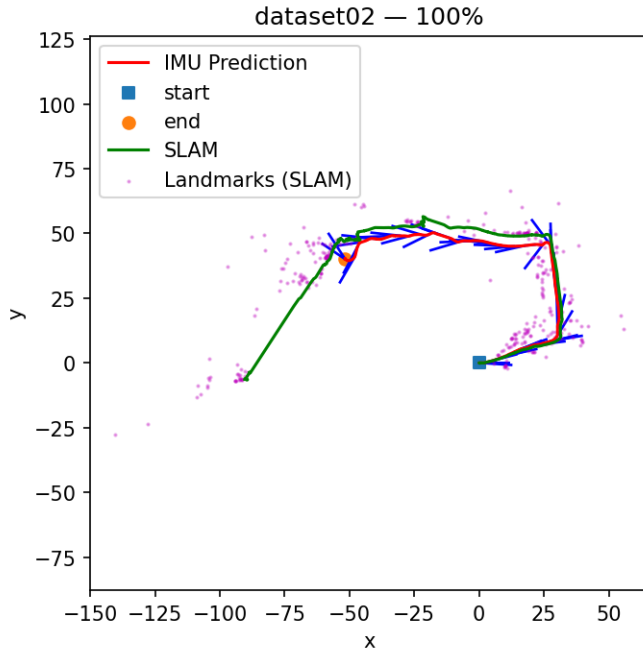


Fig. 12. Dataset 02: VI-SLAM results at 100%.

4) 100%:

D. Discussion

a) *What worked and what did not:* Some methods that worked for me were static point filtering. Since our EKF model assumes that feature points are static, it is critical that we do

this. When watching the videos, I saw that some features that were tracked were from people that were moving, and not all features given are static. By implementing a dynamic point filter, I was able to improve the accuracy of my map output. Furthermore, what ended up doing the most correction for my trajectory was inflating the landmark noise and decreasing the IMU noise. This is because the landmarks are not so reliable due to distance, instability, and other factors, while the IMU trajectory is very reliable as it is mainly on flat ground going along smooth pathways. Also, using the Joseph form of Kalman error correction ended up adding significant numerical stability to my covariance matrix. This, along with the symmetrical adjustment made it so that the covariance matrix did not increase too rapidly, and remained relatively stable.

What didn't work was triangulation through bearing measurements. For some reason, it always produced a far different output than the estimation via. stereo. Therefore, I decided to use stereo triangulation instead. Also, I had to hard-code filtering some residuals that were too large. This mostly happened because the estimated v_R , the vertical pixel in the right camera, was far different from the observed one. This would cause over-correction and distort the overall trajectory.

b) *Results discussion:* The results appear relatively accurate overall, but the estimated trajectory still contains some noticeable drift and jagged edges. These likely occur because some visual features were poorly triangulated and insufficiently filtered before the update step, causing an incorrect correction in the EKF update step. In addition, any drift in the IMU prediction can make these incorrect updates even worse, since the filter may over-correct toward unreliable feature measurements. Also, in dataset02, it was following the correct overall trajectory, but seemed to veer off. The points at that time are very sparse, and so the trajectory may be a result of that sparsity. In conclusion, I was able to recover the general structure of the motion, but the trajectory still jumps and is unstable.

REFERENCES

- [1] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 4: Motion and Observation Models." Accessed: Mar. 13, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_4_MotionAndObservationModels.pdf
- [2] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 6: Localization and Odometry." Accessed: Mar. 13, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_6_LocalizationOdometry.pdf
- [3] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 12: Visual-Inertial SLAM." Accessed: Mar. 20, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_12_VI_SLAM.pdf
- [4] S. Dingler, "Joseph form," *Kalman filter for embedded systems*, Apr. 20, 2025. Accessed: Mar. 21, 2026. [Online]. Available: <https://kalman-filter.com/joseph-form/>